1 ENSAYO

5 / 5

Escriba un algoritmo en **C++** que implemente una **función recursiva** llamada contadorPalabras que reciba como único parámetro una **cadena de texto** y retorne el **número total de palabras** contenidas en la cadena.

Detalles de implementación:

- La función debe identificar palabras separadas por espacios.
- Utilizar **recursividad** para recorrer la cadena sin utilizar bucles explícitos.
- Ignorar múltiples espacios consecutivos.

Ejemplo:

Input:

```
texto = "El esfuerzo de hoy es el éxito de mañana";
//contadorPalabras(texto);
```

Output:

```
9
```

Rúbrica de Calificación:

Criterio	Excelente	Regular	Deficiente
Función contadorPalabras	Implementa correctamente la función recursiva, contando las palabras sin errores (**3 puntos**)	Implementa la función pero con fallos en la lógica recursiva o errores en algunos casos (**1 punto**)	No implementa la función correctamente o no cuenta las palabras adecuadamente (**0 puntos**)
Funcionamiento del programa	El código se ejecuta sin errores, acepta cualquier entrada válida y muestra correctamente el resultado (**2 puntos**)	El programa corre pero presenta fallos en ciertos casos de entrada o errores menores en ejecución (**1 punto**)	El programa no funciona correctamente o tiene errores críticos (**0 puntos**)

2 ENSAYO

15 / 15

La **Caja de Lima** brinda servicio de préstamos a sus diferentes clientes y requiere una aplicación que permita gestionar los registros de los clientes y sus respectivos préstamos.
Se deben generar las clases para clientes y prestamos. Cliente (**Código, Nombre, Apellido, Sexo** (Solo se permite M para Masculino y F para Femenino), **Estado Civil** (Opciones: S para Soltero, C para Casado, V para Viudo, D para Divorciado), **Tipo de Cliente** (Opciones: N para Normal, V para Vip, P para Platinium, O para Oro).
Para los préstamos con la siguiente estructura **Código de préstamo, Cliente asociado, Monto del préstamo, Fecha de registro**

La Caja de Lima desea realizar un **análisis de datos registrados en tiempo real**, por lo que solicita el desarrollo de un **programa que simule la gestión de clientes y préstamos**, asegurando una **estructura eficiente de datos** y una administración adecuada de la información almacenada.
El sistema debe manejar de manera efectiva la información de los clientes y los préstamos, garantizando que los datos ingresados cumplan con las restricciones establecidas.

Para la implementación del sistema, se solicita el uso de **listas enlazadas** con **nodos y clases**. A continuación, se requieren **tres funciones clave** para gestionar la información:

1. Método "**agregarCliente**": Permite registrar un nuevo cliente en la lista enlazada, validando que los datos ingresados cumplan con las restricciones establecidas: Sexo: Solo acepta M (Masculino) o F (Femenino), Estado Civil: Solo acepta S (Soltero), C (Casado), V (Viudo) o D (Divorciado), Tipo de Cliente: Solo acepta N (Normal), V (Vip), P (Platinium) o O (Oro).
2. Método "**contarClientesPorTipo**": Cuenta cuántos clientes están registrados en cada categoría de Tipo de Cliente (Normal, Vip, Platinium, Oro), permitiendo un análisis de la distribución de clientes según su clasificación.
3. Método "**mayorMontoPrimo**": Busca el préstamo con el mayor monto en la lista enlazada y verifica si el valor entero del monto es un número primo. Si el monto es primo, calcula la suma de los dígitos del código de préstamo.
4. Crear un **menú de opciones** para ejecutar cada método de manera interactiva.
5. Para que la solución se considere, los **métodos desarrollados deben estar correctos**, utilizar los **métodos como Nodo** y otros que considere, no debe tener errores y que funcione con cualquier entrada de datos, se **debe validar el ingreso a los datos**. El código debe entenderse y ser claro. No colocar métodos que no se solicitan.

Rúbrica de Calificación

Criterio	Excelente	Regular	Deficiente
Método agregarCliente	Registra correctamente los clientes, validando sexo, estado civil y tipo de cliente sin errores (**2 puntos**)	Registra clientes pero con validaciones incorrectas o errores menores (**1 punto**)	No valida correctamente los datos o no permite agregar clientes (**0 puntos**)
Método contarClientesPorTipo	Cuenta correctamente los clientes por tipo (Normal, Vip, Platinium, Oro) mostrando resultados precisos (**3 puntos**)	Cuenta clientes pero con errores en la comparación o en la estructura de datos (**1 punto**)	No cuenta correctamente los clientes o tiene fallos críticos en la lógica (**0 puntos**)
Método mayorMontoPrimo	Encuentra el préstamo con el mayor monto y verifica correctamente si es primo, calculando la suma de los dígitos del código **(3 puntos)**	Encuentra el mayor monto pero con errores en la verificación de número primo o en la suma de los dígitos **(1 punto)**	No encuentra el mayor monto correctamente o tiene errores críticos en la lógica **(0 puntos)**
Menú de opciones	Implementa un menú interactivo funcional para ejecutar los métodos correctamente (**1 punto**)	Implementa el menú pero con errores menores en la navegación (**0.5 puntos**)	No implementa un menú funcional o tiene errores graves (**0 puntos**)

Uso de clases, listas y nodos	Utiliza correctamente clases, listas y nodos para estructurar la solución de manera eficiente (**4 puntos**)	Usa clases y listas pero con errores en la implementación o falta de estructura clara (**2 puntos**)	No utiliza clases, listas o nodos correctamente, afectando la funcionalidad del programa (**0 puntos**)
Funcionamiento correcto	El programa se ejecuta sin errores, acepta cualquier entrada válida y muestra correctamente los resultados (**2 puntos**)	El programa corre pero presenta fallos en ciertos casos de entrada o errores menores en ejecución (**1 punto**)	El programa no funciona correctamente o tiene errores críticos (**0 puntos**)